

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии
Департамент цифровых, робототехнических систем и электроники

ОТЧЕТ
ПО ПРАКТИЧЕСКОЙ РАБОТЕ №5
дисциплины «Искусственный интеллект и машинное обучение»
Вариант №9

Выполнила:

Ковжого Елизавета Андреевна

2 курс, группа ИВТ-б-о-24-1,

09.03.01 «Информатика и вычислительная техника», направленность (профиль)
«Программное обеспечение средств вычислительной техники и автоматизированных систем», очная форма обучения

(подпись)

Проверил:

Воронкин Р. А., доцент
департамента цифровых,
робототехнических систем и
электроники института
перспективной инженерии.

(подпись)

Отчет защищен с оценкой _____ Дата защиты _____

Ставрополь, 2026 г.

Тема: Введение в pandas: изучение структуры DataFrame и базовые операции.

Цель: познакомиться с основами работы с библиотекой pandas, в частности, со структурой данных DataFrame.

Порядок выполнения работы:

Адрес репозитория: https://github.com/LissKovzogo/AI_ML_LAB_5.git

1. Создали, настроили и клонировали репозиторий AI_ML_LAB_5.
2. Создали виртуальное окружение в VisualStudio Code и создали в JupyterLab.
3. Проработали все примеры лабораторной работы:
Поработали со структурами DataFrame, с их значениями и индексами

```
[2]: data={
      "Возраст": [25, 30, 22],
      "Город": ["Москва", "СПб", "Казань"]
    }
    df = pd.DataFrame(data, index=["Анна", "Иван", "Ольга"])
    print(df)
```

	Возраст	Город
Анна	25	Москва
Иван	30	СПб
Ольга	22	Казань

```
[3]: print(df["Возраст"])
```

Анна	25
Иван	30
Ольга	22

Name: Возраст, dtype: int64

Преобразование Series в DataFrame

```
[4]: s = pd.Series([10, 20, 30], index=['a', 'b', 'c'])
    df = s.to_frame(name='Значение')
    print(df)
```

	Значение
a	10
b	20
c	30

```
[5]: s = df['Значение']
    print(s)
```

a	10
b	20
c	30

Name: Значение, dtype: int64

Рисунок 1. Работа с DataFrame

```
[8]: data={
      "Имя":["Анна","Иван","Ольга"],
      "Возраст":[25,30,22],
      "Город":["Москва","СПб","Казань"]
    }
    df = pd.DataFrame(data)
    print(df)
```

	Имя	Возраст	Город
0	Анна	25	Москва
1	Иван	30	СПб
2	Ольга	22	Казань

```
[9]: ages = pd.Series([25,30,22], index=["a","b","c"])
    cities = pd.Series(["Москва","СПб","Казань"], index=["a","b","c"])
    df= pd.DataFrame({"Возраст": ages, "Город": cities})
    print(df)
```

	Возраст	Город
a	25	Москва
b	30	СПб
c	22	Казань

```
[10]: data = [
      ["Anna",25,"Moscow"],
      ["Ivan",30,"SPb"],
      ["Olga",22,"Kazan"]
    ]
    df = pd.DataFrame(data, columns = ["Name","Age","City"])
    print(df)
```

	Name	Age	City
0	Anna	25	Moscow
1	Ivan	30	SPb
2	Olga	22	Kazan

```
[13]: data = [
      {"Name": "Anna", "Age":25, "City":"Moscow"},
      {"Name":"Ivan", "Age":30},
      {"Name":"Olga", "Age":22, "City":"Kazan"}
    ]
    df = pd.DataFrame(data)
    print(df)
```

	Name	Age	City
0	Anna	25	Moscow
1	Ivan	30	NaN
2	Olga	22	Kazan

Рисунок 2. Работа с DataFrame

```
[3]: import numpy as np
data = np.array([
    ["Anna", 25, "Moskow"],
    ["Ivan", 30, "SPb"],
    ["Olga", 22, "Kazan"]
])
df = pd.DataFrame(data, columns=["Name", "Age", "City"])
print(df)
```

	Name	Age	City
0	Anna	25	Moskow
1	Ivan	30	SPb
2	Olga	22	Kazan

```
[17]: numeric_data = np.array([
    [25, 65.5],
    [30, 78.2],
    [22, 54.3]
])
df = pd.DataFrame(numeric_data, columns=["Age", "Weight"])
print(df)
```

	Age	Weight
0	25.0	65.5
1	30.0	78.2
2	22.0	54.3

```
[18]: df = pd.DataFrame(columns=["Name", "Age", "City"])
print(df)
```

```
Empty DataFrame
Columns: [Name, Age, City]
Index: []
```

```
[19]: df.loc[0]=["Anna", 25, "Moskow"]
df.loc[1]=["Ivan", 30, "SPb"]
print(df)
```

	Name	Age	City
0	Anna	25	Moskow
1	Ivan	30	SPb

Рисунок 3. Работа с DataFrame

```
[24]: df = pd.read_csv("data.csv")
print(df.head())
```

	1	; 72	00	; 1	00	; 80	00	; 47	00	; 6	00	; 25
	\											
0	2	; 48	00	; 92	00	; 82	00	; 37	00	; 13	00	; 27
1	3	; 78	00	; 39	00	; 9	00	; 79	00	; 59	00	; 92
2	4	; 63	00	; 57	00	; 57	00	; 82	00	; 3	00	; 61
3	5	; 61	00	; 5	00	; 57	00	; 15	00	; 98	00	; 58
4	6	; 72	00	; 55	00	; 54	00	; 74	00	; 63	00	; 55
	00	; 13	00	; 69	00	; 3	00	; 89	00	;;;;;;;;;		
0	00	; 15	00	; 100	00	; 50	00	; 19	00	;;;;;;;;;		
1	00	; 56	00	; 23	00	; 1	00	; 92	00	;;;;;;;;;		
2	00	; 25	00	; 74	00	; 54	00	; 72	00	;;;;;;;;;		
3	00	; 43	00	; 76	00	; 7	00	; 79	00	;;;;;;;;;		
4	00	; 98	00	; 20	00	; 70	00	; 96	00	;;;;;;;;;		

Рисунок 4. Работа с DataFrame с csv

```
[33]: import sqlite3
conn = sqlite3.connect("database.db")

conn.execute('''
    CREATE TABLE IF NOT EXISTS users (
        id INTEGER,
        name TEXT,
        email TEXT
    )
''')
conn.execute("INSERT INTO users VALUES (1, 'Alice', 'alice@example.com')")
conn.execute("INSERT INTO users VALUES (2, 'Bob', 'bob@example.com')")
conn.commit()
```

```
[34]: df = pd.read_sql("SELECT * FROM users", conn)
print(df.head())
```

	id	name	email
0	1	Alice	alice@example.com
1	2	Bob	bob@example.com

```
[35]: df.to_sql("users", conn, if_exists="replace", index=False)
```

```
[35]: 2
```

Рисунок 5. Работа с DataFrame с sql

```
[36]: data = {
        "Имя": ["Анна", "Иван", "Ольга", "Петр", "Мария", "Сергей"],
        "Возраст": [25, 30, 22, 40, 35, 28],
        "Город": ["Москва", "СПб", "Казань", "Новосибирск", "Екатеринбург"]
    }
    df = pd.DataFrame(data)
    print(df.head(3))
```

	Имя	Возраст	Город
0	Анна	25	Москва
1	Иван	30	СПб
2	Ольга	22	Казань

```
[37]: print(df.tail(2))
```

	Имя	Возраст	Город
4	Мария	35	Екатеринбург
5	Сергей	28	Сочи

```
[39]: df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 6 entries, 0 to 5
Data columns (total 3 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Имя         6 non-null      str
1   Возраст     6 non-null      int64
2   Город       6 non-null      str
dtypes: int64(1), str(2)
memory usage: 276.0 bytes
```

```
[40]: print(df.describe())
```

	Возраст
count	6.000000
mean	30.000000
std	6.60303
min	22.000000
25%	25.750000
50%	29.000000
75%	33.750000
max	40.000000

Рисунок 6. Работа с DataFrame

```
[42]: data = {
        "Имя": ["Анна", "Иван", "Ольга", "Петр"],
        "Возраст": [25, 30, 22, 40],
        "Город": ["Москва", "СПб", "Казань", "Новосибирск"]
    }

    df = pd.DataFrame(data, index=["a", "b", "c", "d"])
    print(df)
```

	Имя	Возраст	Город
a	Анна	25	Москва
b	Иван	30	СПб
c	Ольга	22	Казань
d	Петр	40	Новосибирск

```
[43]: print(df.loc["c", "Город"])
```

Казань

```
[46]: print(df.loc[["a", "c"]])
```

	Имя	Возраст	Город
a	Анна	25	Москва
c	Ольга	22	Казань

```
[47]: print(df.loc[:, ["Имя", "Город"]])
```

	Имя	Город
a	Анна	Москва
b	Иван	СПб
c	Ольга	Казань
d	Петр	Новосибирск

```
[48]: print(df.loc[df["Возраст"] > 25])
```

	Имя	Возраст	Город
b	Иван	30	СПб
d	Петр	40	Новосибирск

```
[49]: print(df.iloc[1])
```

```
Имя      Иван
Возраст   30
Город     СПб
Name: b, dtype: object
```

```
[50]: print(df.iloc[2,2])
```

Казань

Рисунок 7. Работа с DataFrame

```
[51]: print(df.iloc[[0,2]])
```

	Имя	Возраст	Город
a	Анна	25	Москва
c	Ольга	22	Казань

```
[52]: print(df.iloc[1:3])
```

	Имя	Возраст	Город
b	Иван	30	СПб
c	Ольга	22	Казань

```
[53]: print(df.iloc[:,[0,2]])
```

	Имя	Город
a	Анна	Москва
b	Иван	СПб
c	Ольга	Казань
d	Петр	Новосибирск

```
[54]: print(df.at["c","Город"])
```

Казань

```
[55]: print(df.iat[2,2])
```

Казань

```
[15]: data = {  
    "Имя": ["Анна", "Иван", "Ольга"],  
    "Возраст": [25, 30, 22]  
}  
  
df = pd.DataFrame(data)  
df["Город"] = ["Москва", "СПб", "Казань"]  
print(df)
```

	Имя	Возраст	Город
0	Анна	25	Москва
1	Иван	30	СПб
2	Ольга	22	Казань

```
[17]: df["Страна"]="Россия"  
print(df)
```

	Имя	Возраст	Город	Страна
0	Анна	25	Москва	Россия
1	Иван	30	СПб	Россия
2	Ольга	22	Казань	Россия

Рисунок 8. Работа с DataFrame

```
[11]: df["Возраст в месяцах"] = df["Возраст"] * 12
print(df)
```

	Имя	Возраст	Город	Страна	Возраст в месяцах
0	Анна	25	Москва	Россия	300
1	Иван	30	СПб	Россия	360
2	Ольга	22	Казань	Россия	264

```
[12]: df = df.assign(Зарплата=[50000,60000,45000])
print(df)
```

	Имя	Возраст	Город	Страна	Возраст в месяцах	Зарплата
0	Анна	25	Москва	Россия	300	50000
1	Иван	30	СПб	Россия	360	60000
2	Ольга	22	Казань	Россия	264	45000

```
[13]: df["Категория возраста"] = df["Возраст"].apply(lambda x: "Молодой" if x < 30 else "Взрослый")
print(df)
```

	Имя	Возраст	Город	Страна	Возраст в месяцах	Зарплата	\
0	Анна	25	Москва	Россия	300	50000	
1	Иван	30	СПб	Россия	360	60000	
2	Ольга	22	Казань	Россия	264	45000	

	Категория возраста
0	Молодой
1	Взрослый
2	Молодой

```
[18]: df.loc[3] = ["Петр", 40, "Новосибирск", "Россия"]
print(df)
```

	Имя	Возраст	Город	Страна
0	Анна	25	Москва	Россия
1	Иван	30	СПб	Россия
2	Ольга	22	Казань	Россия
3	Петр	40	Новосибирск	Россия

```
[22]: new_data = pd.DataFrame([{"Имя": "Сергей", "Возраст": 30, "Город": "Сочи"}])
df = pd.concat([df, new_data], ignore_index=True)
print(df)
```

	Имя	Возраст	Город	Страна
0	Анна	25	Москва	Россия
1	Иван	30	СПб	Россия
2	Ольга	22	Казань	Россия
3	Петр	40	Новосибирск	Россия
4	Сергей	30	Сочи	Россия

Рисунок 9. Работа с DataFrame

```
[23]: new_rows = pd.DataFrame([
        {"Имя": "Дмитрий", "Возраст": 31, "Город": "Самара", "Страна": "Р"},
        {"Имя": "Елена", "Возраст": 27, "Город": "Воронеж", "Страна": "Рс"}
    ])

df = pd.concat([df, new_rows], ignore_index=True)
print(df)
```

	Имя	Возраст	Город	Страна
0	Анна	25	Москва	Россия
1	Иван	30	СПб	Россия
2	Ольга	22	Казань	Россия
3	Петр	40	Новосибирск	Россия
4	Сергей	30	Сочи	Россия
5	Дмитрий	31	Самара	Россия
6	Елена	27	Воронеж	Россия

```
[24]: df.loc["новый"]=["Артем",33,"Калуга","Россия"]
print(df)
```

	Имя	Возраст	Город	Страна
0	Анна	25	Москва	Россия
1	Иван	30	СПб	Россия
2	Ольга	22	Казань	Россия
3	Петр	40	Новосибирск	Россия
4	Сергей	30	Сочи	Россия
5	Дмитрий	31	Самара	Россия
6	Елена	27	Воронеж	Россия
новый	Артем	33	Калуга	Россия

```
[27]: data = {
        "Имя": ["Анна", "Иван", "Ольга"],
        "Возраст": [25, 30, 22],
        "Город": ["Москва", "СПб", "Казань"],
        "Зарплата": [50000, 60000, 45000]
    }
df = pd.DataFrame(data)
df = df.drop(columns=["Зарплата"])
print(df)
```

	Имя	Возраст	Город
0	Анна	25	Москва
1	Иван	30	СПб
2	Ольга	22	Казань

```
[26]: df = df.drop(columns=["Возраст", "Город"])
print(df)
```

	Имя
0	Анна
1	Иван
2	Ольга

Рисунок 10. Работа с DataFrame

```
[28]: del df["Имя"]
print(df)
```

	Возраст	Город
0	25	Москва
1	30	СПб
2	22	Казань

```
[29]: df["Имя"] = ["Анна", "Иван", "Ольга"]
удаленный_столбец = df.pop("Имя")
print(удаленный_столбец)
```

0	Анна
1	Иван
2	Ольга

Name: Имя, dtype: str

```
[30]: df = pd.DataFrame(data)
df = df.drop(index=1)
print(df)
```

	Имя	Возраст	Город	Зарплата
0	Анна	25	Москва	50000
2	Ольга	22	Казань	45000

```
[31]: df = pd.DataFrame(data)
df = df.drop(index=[0,2])
print(df)
```

	Имя	Возраст	Город	Зарплата
1	Иван	30	СПб	60000

```
[34]: df = pd.DataFrame(data)
df = df[df["Возраст"]>25]
print(df)
```

	Имя	Возраст	Город	Зарплата
1	Иван	30	СПб	60000

```
[35]: df = df.reset_index(drop=True)
print(df)
```

	Имя	Возраст	Город	Зарплата
0	Иван	30	СПб	60000

```
[36]: df = pd.DataFrame(data)
df.loc[3]=["Мария", None, "Самара", 55000]
df = df.dropna()
print(df)
```

	Имя	Возраст	Город	Зарплата
0	Анна	25	Москва	50000
1	Иван	30	СПб	60000
2	Ольга	22	Казань	45000

Рисунок 11. Работа с DataFrame

```
[37]: df = pd.DataFrame({
        "Имя": ["Анна", "Иван", "Анна", "Ольга"],
        "Возраст": [25, 30, 25, 22]
    })

df = df.drop_duplicates()
print(df)
```

	Имя	Возраст
0	Анна	25
1	Иван	30
3	Ольга	22

```
[49]: data = {
        "Имя": ["Анна", "Иван", "Ольга", "Петр", "Мария"],
        "Возраст": [25, 30, 22, 40, 35],
        "Город": ["Москва", "СПб", "Казань", "Новосибирск", "СПб"],
        "Зарплата": [50000, 60000, 45000, 70000, 65000]
    }

df = pd.DataFrame(data)
print(df)
```

	Имя	Возраст	Город	Зарплата
0	Анна	25	Москва	50000
1	Иван	30	СПб	60000
2	Ольга	22	Казань	45000
3	Петр	40	Новосибирск	70000
4	Мария	35	СПб	65000

```
[44]: df_filt = df.query("Возраст > 30")
print(df_filt)
```

	Имя	Возраст	Город	Зарплата
3	Петр	40	Новосибирск	70000
4	Мария	35	СПб	65000

```
[47]: df_filt = df.query("Город == 'СПб' and Зарплата > 60000")
print(df_filt)
```

	Имя	Возраст	Город	Зарплата
4	Мария	35	СПб	65000

```
[48]: min_age = 25
df_filt = df.query("Возраст > @min_age")
print(df_filt)
```

	Имя	Возраст	Город	Зарплата
1	Иван	30	СПб	60000
3	Петр	40	Новосибирск	70000
4	Мария	35	СПб	65000

Рисунок 12. Работа с DataFrame

```
[60]: data = {
    "Имя": ["Анна", "Иван", "Ольга", "Петр", "Мария"],
    "Возраст": [25, 30, 22, 40, 35],
    "Город": ["Москва", "СПб", "Казань", "Новосибирск", "СПб"],
    "Зарплата": [50000, 60000, 45000, 70000, 65000]
}

df = pd.DataFrame(data)
print(df)
```

	Имя	Возраст	Город	Зарплата
0	Анна	25	Москва	50000
1	Иван	30	СПб	60000
2	Ольга	22	Казань	45000
3	Петр	40	Новосибирск	70000
4	Мария	35	СПб	65000

```
[58]: df_filt = df[df["Город"].isin(["Москва", "СПб"])]
print(df_filt)
```

	Имя	Возраст	Город	Зарплата
0	Анна	25	Москва	50000
1	Иван	30	СПб	60000
4	Мария	35	СПб	65000

```
[61]: df_filt = df[~df["Город"].isin(["Москва", "СПб"])]
print(df_filt)
```

	Имя	Возраст	Город	Зарплата
2	Ольга	22	Казань	45000
3	Петр	40	Новосибирск	70000

```
[63]: df_filt = df[df["Возраст"].between(25, 35)]
print(df_filt)
```

	Имя	Возраст	Город	Зарплата
0	Анна	25	Москва	50000
1	Иван	30	СПб	60000
4	Мария	35	СПб	65000

```
[64]: df_filt = df[df["Возраст"].between(25, 35, inclusive="neither")]
print(df_filt)
```

	Имя	Возраст	Город	Зарплата
1	Иван	30	СПб	60000

Рисунок 13. Работа с DataFrame

```
[67]: data = {
        "Имя": ["Анна", "Иван", "Ольга", "Петр", None],
        "Возраст": [25, 30, 22, None, 35],
        "Город": ["Москва", "СПб", "Казань", "Новосибирск", "СПб"],
    }

    df = pd.DataFrame(data)
    print(df)
```

	Имя	Возраст	Город
0	Анна	25.0	Москва
1	Иван	30.0	СПб
2	Ольга	22.0	Казань
3	Петр	NaN	Новосибирск
4	NaN	35.0	СПб

```
[68]: print(df.count())
```

```
Имя      4
Возраст  4
Город    5
dtype: int64
```

```
[70]: print(df["Город"].value_counts())
```

```
Город
СПб      2
Москва   1
Казань   1
Новосибирск  1
Name: count, dtype: int64
```

```
[71]: print(df["Город"].value_counts(sort=False))
```

```
Город
Москва      1
СПб         2
Казань      1
Новосибирск  1
Name: count, dtype: int64
```

```
[72]: print(df["Имя"].value_counts(dropna=False))
```

```
Имя
Анна      1
Иван      1
Ольга     1
Петр      1
NaN       1
Name: count, dtype: int64
```

Рисунок 14. Работа с DataFrame

```
[73]: print(df.nunique())
```

```
Имя      4
Возраст   4
Город     4
dtype: int64
```

```
[74]: print(df["Имя"].nunique(dropna=False))
```

```
5
```

```
[75]: print(df.isna().sum())
```

```
Имя      1
Возраст   1
Город     0
dtype: int64
```

```
[76]: data = {
    "Имя": ["Анна", "Иван", "Ольга", "Петр", "Мария"],
    "Возраст": [25, 30, 22, 40, 35],
    "Город": ["Москва", "СПб", "Казань", "Новосибирск", "СПб"],
    "Зарплата": [50000, 60000, 45000, 70000, 65000]
}

df = pd.DataFrame(data)
print(df)
```

	Имя	Возраст	Город	Зарплата
0	Анна	25	Москва	50000
1	Иван	30	СПб	60000
2	Ольга	22	Казань	45000
3	Петр	40	Новосибирск	70000
4	Мария	35	СПб	65000

```
[77]: df_sort = df.sort_values(by="Возраст")
print(df_sort)
```

	Имя	Возраст	Город	Зарплата
2	Ольга	22	Казань	45000
0	Анна	25	Москва	50000
1	Иван	30	СПб	60000
4	Мария	35	СПб	65000
3	Петр	40	Новосибирск	70000

```
[79]: df_sort = df.sort_values(by="Возраст", ascending=False)
print(df_sort)
```

	Имя	Возраст	Город	Зарплата
3	Петр	40	Новосибирск	70000
4	Мария	35	СПб	65000
1	Иван	30	СПб	60000
0	Анна	25	Москва	50000
2	Ольга	22	Казань	45000

Рисунок 15. Работа с DataFrame

```
[81]: df_sort = df.sort_values(by=["Возраст", "Зарплата"], ascending=[True,
False])
print(df_sort)
```

	Имя	Возраст	Город	Зарплата
2	Ольга	22	Казань	45000
0	Анна	25	Москва	50000
1	Иван	30	СПб	60000
4	Мария	35	СПб	65000
3	Петр	40	Новосибирск	70000

```
[85]: df.loc[5] = ["Елена", None, "Самара", 55000]
df_sort = df.sort_values(by="Возраст", na_position="first")
print(df_sort)
```

	Имя	Возраст	Город	Зарплата
5	Елена	None	Самара	55000
2	Ольга	22	Казань	45000
0	Анна	25	Москва	50000
1	Иван	30	СПб	60000
4	Мария	35	СПб	65000
3	Петр	40	Новосибирск	70000

```
[86]: df_sort = df.sort_index()
print(df_sort)
```

	Имя	Возраст	Город	Зарплата
0	Анна	25	Москва	50000
1	Иван	30	СПб	60000
2	Ольга	22	Казань	45000
3	Петр	40	Новосибирск	70000
4	Мария	35	СПб	65000
5	Елена	None	Самара	55000

```
[87]: df_sort = df.sort_index(ascending=False)
print(df_sort)
```

	Имя	Возраст	Город	Зарплата
5	Елена	None	Самара	55000
4	Мария	35	СПб	65000
3	Петр	40	Новосибирск	70000
2	Ольга	22	Казань	45000
1	Иван	30	СПб	60000
0	Анна	25	Москва	50000

Рисунок 16. Работа с DataFrame

```
[88]: df = pd.DataFrame({
    "Имя": ["Анна", "Иван", "Ольга"],
    "Возраст": [25, 30, 22],
    "Город": ["Москва", "СПб", "Казань"]
})

print(df)
```

	Имя	Возраст	Город
0	Анна	25	Москва
1	Иван	30	СПб
2	Ольга	22	Казань

```
[89]: from IPython.display import display
display(df)
```

	Имя	Возраст	Город
0	Анна	25	Москва
1	Иван	30	СПб
2	Ольга	22	Казань

```
[94]: data = {
    "Имя": ["Анна", "Иван", "Ольга", "Петр", "Мария", "Мария"],
    "Возраст": [25, 30, 22, 40, 35, 43],
    "Город": ["Москва", "СПб", "Казань", "Новосибирск", "СПб", "Новосибирск"],
    "Зарплата": [50000, 60000, 45000, 70000, 65000, 90000]
}

df = pd.DataFrame(data)
df.head(3)
```

```
[94]:
```

	Имя	Возраст	Город	Зарплата
0	Анна	25	Москва	50000
1	Иван	30	СПб	60000
2	Ольга	22	Казань	45000

Рисунок 17. Работа с DataFrame

```
[93]: df.tail(3)
```

```
[93]:
```

	Имя	Возраст	Город	Зарплата
3	Петр	40	Новосибирск	70000
4	Мария	35	СПб	65000
5	Мария	43	Новосибирск	90000

```
[95]: pd.set_option("display.max_rows", 100)
```

```
[96]: pd.set_option("display.max_columns", 50)
```

```
[97]: pd.set_option("display.max_colwidth", None)
```

```
[98]: pd.set_option("display.float_format", "{:.2f}".format)#отображение чи
```

```
[100]: df = pd.DataFrame({
    "Имя": ["Анна", "Иван", "Ольга"],
    "Возраст": [25, 30, 22],
    "Город": ["Москва", "СПб", "Казань"]
})
df.style.set_properties(**{"background-color": "lightgray", "color":
html_table = df.to_html()
with open("table.html", "w", encoding="utf-8") as f:
    f.write(html_table)
```

	Имя	Возраст	Город
0	Анна	25	Москва
1	Иван	30	СПб
2	Ольга	22	Казань

```
[104]: styled_df = df.style.set_properties(**{"background-color": "lightgray"
display(styled_df)
```

	Имя	Возраст	Город
0	Анна	25	Москва
1	Иван	30	СПб
2	Ольга	22	Казань

Рисунок 18. Работа с DataFrame

2.Выполнили представленные задания:

1. Создание DataFrame разными способами

1. Создайте DataFrame из словаря списков с данными о пяти сотрудниках (возьмите из таблицы 1).
2. Создайте DataFrame из списка словарей с теми же данными.
3. Создайте DataFrame из массива NumPy, заполненного случайными числами от 20 до 60 (для возраста сотрудников).
4. Проверьте типы данных в каждом столбце с помощью .info().

Создание DataFrame разными способами

```
[8]: import pandas as pd
import numpy as np

data_dict = {
    "ID": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20],
    "Имя": ["Иван", "Ольга", "Алексей", "Мария", "Сергей", "Анна", "Дмитрий", "Елена", "Виктор", "Алиса",
    "Лет": [25, 30, 40, 35, 28, 32, 45, 29, 31, 27, 33, 26, 42, 37, 39, 24, 50, 45, 41, 38],
    "Дол-ть": ["Инженер", "Аналитик", "Менеджер", "Программист", "Специалист", "Разработчик", "HR", "Мар",
    "Отдел": ["IT", "Маркетинг", "Продажи", "IT", "HR", "IT", "HR", "Маркетинг", "Юридический", "Дизайн",
    "Зп": [60000, 75000, 90000, 80000, 50000, 85000, 48000, 70000, 95000, 62000, 55000, 67000, 105000, 7
    "Стаж": [2, 5, 15, 7, 3, 6, 12, 4, 10, 5, 7, 2, 20, 9, 11, 3, 25, 20, 14, 8]
}

df1 = pd.DataFrame(data_dict)
print(df1)
```

	ID	Имя	Лет	Дол-ть	Отдел	Зп	Стаж
0	1	Иван	25	Инженер	IT	60000	2
1	2	Ольга	30	Аналитик	Маркетинг	75000	5
2	3	Алексей	40	Менеджер	Продажи	90000	15
3	4	Мария	35	Программист	IT	80000	7
4	5	Сергей	28	Специалист	HR	50000	3
5	6	Анна	32	Разработчик	IT	85000	6
6	7	Дмитрий	45	HR	HR	48000	12
7	8	Елена	29	Маркетолог	Маркетинг	70000	4
8	9	Виктор	31	Юрист	Юридический	95000	10
9	10	Алиса	27	Дизайнер	Дизайн	62000	5
10	11	Павел	33	Администратор	Администрация	55000	7
11	12	Светлана	26	Тестировщик	Тестирование	67000	2
12	13	Роман	42	Финансист	Финансы	105000	20
13	14	Татьяна	37	Редактор	Редакция	72000	9
14	15	Николай	39	Логист	Логистика	75000	11
15	16	Валерия	24	SEO-специалист	SEO	64000	3
16	17	Григорий	50	Бухгалтер	Бухгалтерия	110000	25
17	18	Юлия	45	Директор	Финансы	150000	20
18	19	Степан	41	Экономист	Экономика	98000	14
19	20	Василиса	38	Проект-менеджер	Продажи	88000	8

Рисунок 2. Задание №1


```
[3]: data_list = [
    {"ID": 1, "Имя": "Иван", "Возраст": 25, "Должность": "Инженер", "Отдел": "IT", "Зарплата": 60000, "Статус": "Активен"},
    {"ID": 2, "Имя": "Ольга", "Возраст": 30, "Должность": "Аналитик", "Отдел": "Маркетинг", "Зарплата": 75000, "Статус": "Активен"},
    {"ID": 3, "Имя": "Алексей", "Возраст": 40, "Должность": "Менеджер", "Отдел": "Продажи", "Зарплата": 90000, "Статус": "Активен"},
    {"ID": 4, "Имя": "Мария", "Возраст": 35, "Должность": "Программист", "Отдел": "IT", "Зарплата": 80000, "Статус": "Активен"},
    {"ID": 5, "Имя": "Сергей", "Возраст": 28, "Должность": "Специалист", "Отдел": "HR", "Зарплата": 50000, "Статус": "Активен"},
    {"ID": 6, "Имя": "Анна", "Возраст": 32, "Должность": "Разработчик", "Отдел": "IT", "Зарплата": 85000, "Статус": "Активен"},
    {"ID": 7, "Имя": "Дмитрий", "Возраст": 45, "Должность": "HR", "Отдел": "HR", "Зарплата": 48000, "Статус": "Активен"},
    {"ID": 8, "Имя": "Елена", "Возраст": 29, "Должность": "Маркетолог", "Отдел": "Маркетинг", "Зарплата": 70000, "Статус": "Активен"},
    {"ID": 9, "Имя": "Виктор", "Возраст": 31, "Должность": "Юрист", "Отдел": "Юридический", "Зарплата": 95000, "Статус": "Активен"},
    {"ID": 10, "Имя": "Алиса", "Возраст": 27, "Должность": "Дизайнер", "Отдел": "Дизайн", "Зарплата": 62000, "Статус": "Активен"},
    {"ID": 11, "Имя": "Павел", "Возраст": 33, "Должность": "Администратор", "Отдел": "Администрация", "Зарплата": 55000, "Статус": "Активен"},
    {"ID": 12, "Имя": "Светлана", "Возраст": 26, "Должность": "Тестировщик", "Отдел": "Тестирование", "Зарплата": 67000, "Статус": "Активен"},
    {"ID": 13, "Имя": "Роман", "Возраст": 42, "Должность": "Финансист", "Отдел": "Финансы", "Зарплата": 105000, "Статус": "Активен"},
    {"ID": 14, "Имя": "Татьяна", "Возраст": 37, "Должность": "Редактор", "Отдел": "Редакция", "Зарплата": 72000, "Статус": "Активен"},
    {"ID": 15, "Имя": "Николай", "Возраст": 39, "Должность": "Логист", "Отдел": "Логистика", "Зарплата": 75000, "Статус": "Активен"},
    {"ID": 16, "Имя": "Валерия", "Возраст": 24, "Должность": "SEO-специалист", "Отдел": "SEO", "Зарплата": 64000, "Статус": "Активен"},
    {"ID": 17, "Имя": "Григорий", "Возраст": 50, "Должность": "Бухгалтер", "Отдел": "Бухгалтерия", "Зарплата": 110000, "Статус": "Активен"},
    {"ID": 18, "Имя": "Юлия", "Возраст": 45, "Должность": "Директор", "Отдел": "Финансы", "Зарплата": 150000, "Статус": "Активен"},
    {"ID": 19, "Имя": "Степан", "Возраст": 41, "Должность": "Экономист", "Отдел": "Экономика", "Зарплата": 98000, "Статус": "Активен"},
    {"ID": 20, "Имя": "Василиса", "Возраст": 38, "Должность": "Проект-менеджер", "Отдел": "Продажи", "Зарплата": 88000, "Статус": "Активен"}
]

df2 = pd.DataFrame(data_list)
print(df2)
```

	ID	Имя	Возраст	Должность	Отдел	Зарплата \
0	1	Иван	25	Инженер	IT	60000
1	2	Ольга	30	Аналитик	Маркетинг	75000
2	3	Алексей	40	Менеджер	Продажи	90000
3	4	Мария	35	Программист	IT	80000
4	5	Сергей	28	Специалист	HR	50000
5	6	Анна	32	Разработчик	IT	85000
6	7	Дмитрий	45	HR	HR	48000
7	8	Елена	29	Маркетолог	Маркетинг	70000
8	9	Виктор	31	Юрист	Юридический	95000
9	10	Алиса	27	Дизайнер	Дизайн	62000
10	11	Павел	33	Администратор	Администрация	55000
11	12	Светлана	26	Тестировщик	Тестирование	67000
12	13	Роман	42	Финансист	Финансы	105000
13	14	Татьяна	37	Редактор	Редакция	72000
14	15	Николай	39	Логист	Логистика	75000
15	16	Валерия	24	SEO-специалист	SEO	64000
16	17	Григорий	50	Бухгалтер	Бухгалтерия	110000
17	18	Юлия	45	Директор	Финансы	150000
18	19	Степан	41	Экономист	Экономика	98000
19	20	Василиса	38	Проект-менеджер	Продажи	88000

Рисунок 20. Задание №1

```
[4]: ages = np.random.randint(20, 61, size=20)
df3 = pd.DataFrame({
    "ID": range(1, 21),
    "Возраст": ages
})
print(df3)
```

	ID	Возраст
0	1	27
1	2	20
2	3	45
3	4	55
4	5	52
5	6	60
6	7	60
7	8	36
8	9	34
9	10	48
10	11	20
11	12	25
12	13	52
13	14	34
14	15	37
15	16	25
16	17	48
17	18	40
18	19	59
19	20	49

```
[9]: print("df1")
df1.info()

print("\ndf2")
df2.info()

print("\ndf3")
df3.info()
```

```
df1
<class 'pandas.DataFrame'>
RangeIndex: 20 entries, 0 to 19
Data columns (total 7 columns):
#   Column   Non-Null Count  Dtype
---  -
0    ID       20 non-null    int64
1    Имя      20 non-null    str
2    Лет      20 non-null    int64
3    Дол-ть   20 non-null    str
4    Отдел    20 non-null    str
5    Зп       20 non-null    int64
6    Стаж     20 non-null    int64
..
```

Рисунок 21. Задание №1

2. Чтение данных из файлов (CSV, Excel, JSON)

Используйте предоставленные таблицы и сохраните их в файлы перед чтением.

1. Сохраните Таблицу 1 в CSV и затем загрузите ее обратно в DataFrame.

2. Запишите Таблицу 2 в Excel (data.xlsx, лист "Клиенты") и прочитайте ее в DataFrame.

3. Экспортируйте Таблицу 1 в формат JSON и затем прочитайте ее обратно в DataFrame.

```
[3]: import pandas as pd
data_table1 = {
    "ID": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20],
    "Имя": ["Иван", "Ольга", "Алексей", "Мария", "Сергей", "Анна", "Дмитрий", "Елена", "Виктор", "Алиса"],
    "Возраст": [25, 30, 40, 35, 28, 32, 45, 29, 31, 27, 33, 26, 42, 37, 39, 24, 50, 45, 41, 38],
    "Должность": ["Инженер", "Аналитик", "Менеджер", "Программист", "Специалист", "Разработчик", "HR", "Маркетинг", "Продажи", "Юридический", "Дизайн"],
    "Отдел": ["IT", "Маркетинг", "Продажи", "IT", "HR", "IT", "HR", "Маркетинг", "Продажи", "Юридический", "Дизайн"],
    "Зарплата": [60000, 75000, 90000, 80000, 50000, 85000, 48000, 70000, 95000, 62000, 55000, 67000, 105000, 78000, 92000, 81000, 69000, 73000, 86000, 91000],
    "Стаж": [2, 5, 15, 7, 3, 6, 12, 4, 10, 5, 7, 2, 20, 9, 11, 3, 25, 20, 14, 8]
}

df1 = pd.DataFrame(data_table1)

df1.to_csv("table1.csv", index=False, encoding="utf-8")

df1_loaded = pd.read_csv("table1.csv", encoding="utf-8")

print(df1_loaded.head())
```

	ID	Имя	Возраст	Должность	Отдел	Зарплата	Стаж
0	1	Иван	25	Инженер	IT	60000	2
1	2	Ольга	30	Аналитик	Маркетинг	75000	5
2	3	Алексей	40	Менеджер	Продажи	90000	15
3	4	Мария	35	Программист	IT	80000	7
4	5	Сергей	28	Специалист	HR	50000	3

```
[4]: data_table2 = {
    "ID": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20],
    "Имя": ["Иван", "Ольга", "Алексей", "Мария", "Сергей", "Анна", "Дмитрий", "Елена", "Виктор", "Алиса"],
    "Возраст": [34, 27, 45, 38, 29, 50, 31, 40, 28, 33, 46, 37, 41, 25, 39, 42, 49, 50, 30, 35],
    "Город": ["Москва", "Санкт-Петербург", "Казань", "Новосибирск", "Екатеринбург", "Воронеж", "Челябинск", "Самара", "Уфа", "Тюмень", "Ярославль", "Ижевск", "Владивосток", "Хабаровск", "Красноярск", "Новосибирск", "Екатеринбург", "Воронеж", "Челябинск", "Самара", "Уфа"],
    "Баланс": [120000, 80000, 150000, 200000, 95000, 300000, 140000, 175000, 110000, 98000, 250000, 210000, 180000, 160000, 190000, 220000, 130000, 110000, 140000, 170000],
    "Кред. история": ["Хорошая", "Средняя", "Плохая", "Хорошая", "Средняя", "Отличная", "Средняя", "Хорошая", "Средняя", "Плохая", "Хорошая", "Средняя", "Отличная", "Средняя", "Хорошая", "Средняя", "Плохая", "Хорошая", "Средняя", "Отличная"]
}

df2 = pd.DataFrame(data_table2)

with pd.ExcelWriter("data.xlsx") as writer:
    df2.to_excel(writer, sheet_name="Клиенты", index=False)

df2_loaded = pd.read_excel("data.xlsx", sheet_name="Клиенты")

print(df2_loaded.head())
```

	ID	Имя	Возраст	Город	Баланс	Кред. история
0	1	Иван	34	Москва	120000	Хорошая
1	2	Ольга	27	Санкт-Петербург	80000	Средняя
2	3	Алексей	45	Казань	150000	Плохая
3	4	Мария	38	Новосибирск	200000	Хорошая
4	5	Сергей	29	Екатеринбург	95000	Средняя

Рисунок 22. Задание №2

```
[5]: df1.to_json("table1.json", orient="records", force_ascii=False, indent=2)

df1_loaded = pd.read_json("table1.json", orient="records")

print(df1_loaded.head())
```

	ID	Имя	Возраст	Должность	Отдел	Зарплата	Стаж
0	1	Иван	25	Инженер	IT	60000	2
1	2	Ольга	30	Аналитик	Маркетинг	75000	5
2	3	Алексей	40	Менеджер	Продажи	90000	15
3	4	Мария	35	Программист	IT	80000	7
4	5	Сергей	28	Специалист	HR	50000	3

Рисунок 23. Задание №2

3. Доступ к данным (.loc, .iloc, .at, .iat)

1. Получите информацию о сотруднике с ID = 5 с помощью .loc[].
2. Выведите возраст третьего сотрудника в таблице с .iloc[].
3. Выведите название отдела для сотрудника "Мария" с .at[].
4. Выведите зарплату сотрудника, находящегося в четвертой строке и пятом столбце, используя .iat[].

Доступ к данным

```
[4]: import pandas as pd
data_table1 = {
    "ID": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20],
    "Имя": ["Иван", "Ольга", "Алексей", "Мария", "Сергей", "Анна", "Дмитрий", "Елена", "Александр", "Екатерина", "Андрей", "Ольга", "Александр", "Екатерина", "Андрей", "Ольга", "Александр", "Екатерина", "Андрей", "Ольга"],
    "Возраст": [25, 30, 40, 35, 28, 32, 45, 29, 31, 27, 33, 26, 42, 38, 34, 29, 36, 31, 28, 35],
    "Должность": ["Инженер", "Аналитик", "Менеджер", "Программист", "Специалист", "Менеджер", "Программист", "Специалист", "Менеджер", "Программист", "Специалист", "Менеджер", "Программист", "Специалист", "Менеджер", "Программист", "Специалист", "Менеджер", "Программист", "Специалист"],
    "Отдел": ["IT", "Маркетинг", "Продажи", "IT", "HR", "IT", "HR", "IT", "Маркетинг", "Продажи", "IT", "HR", "IT", "Маркетинг", "Продажи", "IT", "HR", "IT", "Маркетинг", "Продажи"],
    "Зарплата": [60000, 75000, 90000, 80000, 50000, 85000, 48000, 70000, 65000, 55000, 80000, 72000, 68000, 58000, 82000, 74000, 69000, 59000, 81000, 73000],
    "Стаж": [2, 5, 15, 7, 3, 6, 12, 4, 10, 5, 7, 2, 20, 9, 11, 3, 25, 8, 6, 14]
}

df = pd.DataFrame(data_table1)
df1 = df.set_index("ID")
print(f"Сотрудник с 5-м индексом:\n{df1.loc[5]}")
```

Сотрудник с 5-м индексом:

Имя	Сергей
Возраст	28
Должность	Специалист
Отдел	HR
Зарплата	50000
Стаж	3

Name: 5, dtype: object

```
[5]: print(f"Возраст третьего сотрудника:{df.iloc[2]["Возраст"]}")
```

Возраст третьего сотрудника:40

```
[7]: df2 = df.set_index("Имя")
print(f"Отдел сотрудника Марии: {df2.at["Мария", "Отдел"]}")
```

Отдел сотрудника Марии: IT

```
[8]: print(f"Зарплата 4-го сотрудника:{df.iat[3, 5]}")
```

Зарплата 4-го сотрудника:80000

Рисунок 24. Задание №3

4. Добавление новых столбцов и строк

Используя Таблицу 1:

1. Добавьте новый столбец "Категория зарплаты".
2. Добавьте нового сотрудника.
5. Удаление строк и столбцов

1. Удалите столбец "Категория зарплаты" с `.drop()`.
2. Удалите строку с `ID = 10`.
3. Удалите все строки, где Стаж работы < 3 лет.
4. Удалите все столбцы, кроме Имя, Должность, Зарплата.

Добавление новых столбцов и строк

```
2]: import pandas as pd
data_table1 = {
    "ID": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20],
    "Имя": ["Иван", "Ольга", "Алексей", "Мария", "Сергей", "Анна", "Дмитрий", "Елена", "Александр", "Екатерина"],
    "Возраст": [25, 30, 40, 35, 28, 32, 45, 29, 31, 27, 33, 26, 42, 38, 34, 29, 36, 39, 37, 30],
    "Должность": ["Инженер", "Аналитик", "Менеджер", "Программист", "Маркетолог", "HR", "Юрист", "Дизайнер", "Менеджер", "Аналитик"],
    "Отдел": ["IT", "Маркетинг", "Продажи", "IT", "HR", "IT", "HR", "Продажи", "Маркетинг", "IT"],
    "Зарплата": [60000, 75000, 90000, 80000, 50000, 85000, 48000, 70000, 65000, 95000],
    "Стаж": [2, 5, 15, 7, 3, 6, 12, 4, 10, 5, 7, 2, 20, 9, 11, 3, 25, 8, 14]}
}
```

```
df = pd.DataFrame(data_table1)
```

```
6]: df["Категория зарплаты"] = ["Низкая" if x < 60000 else "Высокая" for
```

```
7]: df.loc[20] = [21, "Антон", 32, "Разработчик", "IT", 85000, 6, "Высокая"]
```

```
8]: new = pd.DataFrame([
    [22, "Максим", 29, "Аналитик", "Аналитика", 70000, 4, "Высокая"],
    [23, "Ксения", 34, "Менеджер", "Продажи", 95000, 9, "Высокая"]
], columns=df.columns)

df = pd.concat([df, new], ignore index=True)
```

Рисунок 25. Задание №4

Удаление строк и столбцов

```
l0]: df = df.drop(columns=["Категория зарплаты"])
print(df.head())
```

	ID	Имя	Возраст	Должность	Отдел	Зарплата	Стаж
0	1	Иван	25	Инженер	IT	60000	2
1	2	Ольга	30	Аналитик	Маркетинг	75000	5
2	3	Алексей	40	Менеджер	Продажи	90000	15
3	4	Мария	35	Программист	IT	80000	7
4	5	Сергей	28	Специалист	HR	50000	3

```
l3]: df = df[df["ID"] != 10]
print(df.head(10))
```

	ID	Имя	Возраст	Должность	Отдел	Зарплата	Стаж
0	1	Иван	25	Инженер	IT	60000	2
1	2	Ольга	30	Аналитик	Маркетинг	75000	5
2	3	Алексей	40	Менеджер	Продажи	90000	15
3	4	Мария	35	Программист	IT	80000	7
4	5	Сергей	28	Специалист	HR	50000	3
5	6	Анна	32	Разработчик	IT	85000	6
6	7	Дмитрий	45	HR	HR	48000	12
7	8	Елена	29	Маркетолог	Маркетинг	70000	4
8	9	Виктор	31	Юрист	Юридический	95000	10
10	11	Павел	33	Администратор	Администрация	55000	7

```
l5]: df = df[df["Стаж"] >= 3]
print(df)
```

Рисунок 26. Задание №5

6. Фильтрация данных (query, isin, between)

1. Выберите всех клиентов из "Москва" или "Санкт-Петербург", используя .isin().

2. Выберите клиентов, у которых Балланс на счете от 100000 до 250000, используя `.between()`.

3. Отфильтруйте клиентов, у которых "Кредитная история" "Хорошая" и "Балланс на счете" > 150000 , используя `.query()`.

```
[7]: import pandas as pd

data = {
    "ID": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19],
    "Имя": ["Иван", "Ольга", "Алексей", "Мария", "Сергей", "Анна", "Дмитрий", "Елена", "Виктор", "Павел", "Светлана", "Роман", "Татьяна", "Николай", "Валерия", "Степан"],
    "Возраст": [34, 27, 45, 38, 29, 50, 31, 40, 28, 33, 46, 37, 41, 25, 39, 42, 30],
    "Город": ["Москва", "Санкт-Петербург", "Казань", "Новосибирск", "Челябинск", "Краснодар", "Ростов-на-Дону", "Омск", "Пермь", "Тюмень", "Саратов", "Самара", "Волгоград", "Хабаровск"],
    "Баланс": [120000, 80000, 150000, 200000, 95000, 300000, 140000, 175000, 110000, 250000, 210000, 135000, 155000, 125000, 180000, 105000],
    "Кред.история": ["Хорошая", "Средняя", "Плохая", "Хорошая", "Средняя", "Хорошая", "Плохая", "Хорошая", "Плохая", "Хорошая", "Отличная", "Средняя", "Хорошая", "Средняя", "Плохая", "Средняя"]
}
df = pd.DataFrame(data)
```

```
[8]: cities = ["Москва", "Санкт-Петербург"]
filter1 = df[df["Город"].isin(cities)]
print(f"Клиенты из Москвы или Санкт-Петербурга (.isin()):\n")
print(filter1)
```

Клиенты из Москвы или Санкт-Петербурга (.isin()):

	ID	Имя	Возраст	Город	Баланс	Кред.история
0	1	Иван	34	Москва	120000	Хорошая
1	2	Ольга	27	Санкт-Петербург	80000	Средняя

```
10]: filter2 = df[df["Баланс"].between(100000, 250000)]
print("Клиенты с балансом от 100000 до 250000 (.between()):")
print(filter2)
```

Клиенты с балансом от 100000 до 250000 (.between()):

	ID	Имя	Возраст	Город	Баланс	Кред.история
0	1	Иван	34	Москва	120000	Хорошая
2	3	Алексей	45	Казань	150000	Плохая
3	4	Мария	38	Новосибирск	200000	Хорошая
6	7	Дмитрий	31	Челябинск	140000	Средняя
7	8	Елена	40	Краснодар	175000	Хорошая
8	9	Виктор	28	Ростов-на-Дону	110000	Плохая
10	11	Павел	46	Омск	250000	Хорошая
11	12	Светлана	37	Пермь	210000	Отличная
12	13	Роман	41	Тюмень	135000	Средняя
13	14	Татьяна	25	Саратов	155000	Хорошая
14	15	Николай	39	Самара	125000	Средняя
15	16	Валерия	42	Волгоград	180000	Плохая
18	19	Степан	30	Хабаровск	105000	Средняя

Рисунок 27. Задание №6

```
[17]: filter3 = df.query('~Кред.история` == "Хорошая" and Баланс > 150000')
print("Клиенты с кредитной историей 'Хорошая' и балансом > 150000 .qu
print(filter3)
```

Клиенты с кредитной историей 'Хорошая' и балансом > 150000 .query():

	ID	Имя	Возраст	Город	Баланс	Кред.история
3	4	Мария	38	Новосибирск	200000	Хорошая
7	8	Елена	40	Краснодар	175000	Хорошая
10	11	Павел	46	Омск	250000	Хорошая
13	14	Татьяна	25	Саратов	155000	Хорошая
17	18	Юлия	50	Иркутск	320000	Хорошая

Рисунок 28. Задание №6

7. Подсчет значений (count, value_counts, unique)

1. Подсчитайте количество непустых значений в каждом столбце (.count()).
2. Определите частоту встречаемости значений в "Город" (.value_counts()).
3. Найдите количество уникальных значений в "Город", "Возраст", "Баланс на счете" (.unique()).

Подсчет значений

```
[1]: import pandas as pd

data = {
    "ID": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20],
    "Имя": ["Иван", "Ольга", "Алексей", "Мария", "Сергей", "Анна", "Дмитрий", "Елена", "Александр", "Екатерина", "Андрей", "Ольга", "Александр", "Екатерина", "Андрей", "Ольга", "Александр", "Екатерина", "Андрей", "Ольга"],
    "Возраст": [34, 27, 45, 38, 29, 50, 31, 40, 28, 33, 46, 37, 41, 29, 35, 42, 36, 43, 39, 32],
    "Город": ["Москва", "Санкт-Петербург", "Казань", "Новосибирск", "Екатеринбург", "Самара", "Уфа", "Волгоград", "Красноярск", "Иркутск", "Хабаровск", "Магнитогорск", "Тюмень", "Омск", "Барнаул", "Кемерово", "Липецк", "Воронеж", "Брянск", "Пенза"],
    "Баланс": [120000, 80000, 150000, 200000, 95000, 300000, 140000, 110000, 160000, 130000, 180000, 150000, 190000, 170000, 210000, 140000, 120000, 160000, 130000, 170000],
    "Кред.история": ["Хорошая", "Средняя", "Плохая", "Хорошая", "Средняя", "Плохая", "Хорошая", "Средняя", "Плохая", "Хорошая", "Средняя", "Плохая", "Хорошая", "Средняя", "Плохая", "Хорошая", "Средняя", "Плохая", "Хорошая", "Средняя"]
}

df = pd.DataFrame(data)

[2]: print("1. Количество непустых значений .count():")
print(df.count())

1. Количество непустых значений .count():
ID                20
Имя               20
Возраст           20
Город             20
Баланс            20
Кред.история      20
dtype: int64

[9]: print("Частота встречаемости городов .value_counts():")
city_counts = df["Город"].value_counts()
print(city_counts)
print(len(city_counts))
```

Рисунок 29. Задание №7

```
[7]: print(f"Города: {df['Город'].nunique()}")
print(f"Возраста: {df['Возраст'].nunique()}")
print(f"Баланс на счете: {df['Баланс'].nunique()}")

Города: 20
Возраста: 19
Баланс на счете: 20
```

Рисунок 30. Задание №7

8. Обнаружение пропусков (isna, notna)

1. Подсчитайте количество NaN в каждом столбце (`.isna().sum()`).
2. Подсчитайте количество заполненных значений в каждом столбце (`.notna().sum()`).
3. Выведите DataFrame, оставив только строки, где нет пропущенных значений.

Обнаружение пропусков










5]:

```

import pandas as pd
import numpy as np

# Таблица 3 (с пропусками)
data = {
    "ID": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20],
    "Имя": ["Иван", "Ольга", "Алексей", "Мария", "Сергей", np.nan, "Дмитрий", "Елена", "Виктор", "Алиса", "Павел", "NaN", "Роман", "Татьяна", "Николай", "Валерия", "Григорий", "Юлия", "Степан", "Василиса"],
    "Возраст": [34.0, 27.0, np.nan, 38.0, 29.0, 50.0, 31.0, 40.0, np.nan, 33.0, 46.0, 37.0, 41.0, 25.0, np.nan, 42.0, 49.0, np.nan, 30.0, 35.0],
    "Город": ["Москва", "Санкт-Петербург", "Казань", np.nan, "Екатеринбург", "Воронеж", np.nan, "Краснодар", "Ростов-на-Дону", "Уфа", "Омск", "Пермь", np.nan, "Саратов", "Самара", np.nan, "Барнаул", "Иркутск", "Хабаровск", "Томск"],
    "Баланс": [120000.0, np.nan, 150000.0, 200000.0, np.nan, 300000.0, 140000.0, 175000.0, 110000.0, np.nan, 250000.0, 210000.0, 135000.0, 155000.0, 125000.0, np.nan, 275000.0, 320000.0, 105000.0, 90000.0],
    "Кредитная": ["Хорошая", "Средняя", "Плохая", "Хорошая", np.nan, "Отличная", "Средняя", "Хорошая", np.nan, "Средняя", "Хорошая", "Отличная", "Средняя", np.nan, "Средняя", "Плохая", "Отличная", "Хорошая", "Средняя", "Плохая"]
}

df = pd.DataFrame(data)
print(df)

```

	ID	Имя	Возраст	Город	Баланс	Кредитная
0	1	Иван	34.0	Москва	120000.0	Хорошая
1	2	Ольга	27.0	Санкт-Петербург	NaN	Средняя
2	3	Алексей	NaN	Казань	150000.0	Плохая
3	4	Мария	38.0	NaN	200000.0	Хорошая
4	5	Сергей	29.0	Екатеринбург	NaN	NaN
5	6	NaN	50.0	Воронеж	300000.0	Отличная
6	7	Дмитрий	31.0	NaN	140000.0	Средняя
7	8	Елена	40.0	Краснодар	175000.0	Хорошая
8	9	Виктор	NaN	Ростов-на-Дону	110000.0	NaN
9	10	Алиса	33.0	Уфа	NaN	Средняя
10	11	Павел	46.0	Омск	250000.0	Хорошая
11	12	NaN	37.0	Пермь	210000.0	Отличная
12	13	Роман	41.0	NaN	135000.0	Средняя
13	14	Татьяна	25.0	Саратов	155000.0	NaN
14	15	Николай	NaN	Самара	125000.0	Средняя
15	16	Валерия	42.0	NaN	NaN	Плохая
16	17	Григорий	49.0	Барнаул	275000.0	Отличная
17	18	Юлия	NaN	Иркутск	320000.0	Хорошая
18	19	Степан	30.0	Хабаровск	105000.0	Средняя
19	20	Василиса	35.0	Томск	90000.0	Плохая

Рисунок 31. Задание №8

19 20 Василиса 35.0 Томск 90000.0 Плохая

```
[6]: print("Количество пропусков по столбцам:")
nan_counts = df.isna().sum()
print(nan_counts)
```

1. Количество пропусков по столбцам:

```
ID          0
Имя         2
Возраст     4
Город       4
Баланс      4
Кредитная  3
dtype: int64
```

```
[7]: print("Количество заполненных значений:")
notna_counts = df.notna().sum()
print(notna_counts)
```

Количество заполненных значений:

```
ID          20
Имя         18
Возраст     16
Город       16
Баланс      16
Кредитная  17
dtype: int64
```

```
[8]: df_clean = df.dropna()
print("DataFrame без пропусков:")
print(df_clean)
```

DataFrame без пропусков:

	ID	Имя	Возраст	Город	Баланс	Кредитная
0	1	Иван	34.0	Москва	120000.0	Хорошая
7	8	Елена	40.0	Краснодар	175000.0	Хорошая
10	11	Павел	46.0	Омск	250000.0	Хорошая
16	17	Григорий	49.0	Барнаул	275000.0	Отличная
18	19	Степан	30.0	Хабаровск	105000.0	Средняя
19	20	Василиса	35.0	Томск	90000.0	Плохая

[1]:

Рисунок 32. Задание №8

Индивидуальное задание:

Общие требования:

Написать программу на языке программирования Python для решения поставленной задачи (в репозитории должны присутствовать настройки требуемых пакетов для выбранного менеджера пакетов). Приложение должно использовать интерфейс командной строки (модуль `argparse`) или графический интерфейс пользователя (модули `tkinter`, `PySide2`, `Kivy` и т.д.). При работе с датой и временем использовать пакет `datetime`. Организовать чтение и сохранение данных из/в формат `Parquet`. Выполнить валидацию сохраненных данных с помощью сторонних приложений для работы с форматом `Parquet`. Организовать также удаление данных по одной из колонок `DataFrame` на выбор обучающихся. Номер варианта определяется по согласованию с преподавателем.

Условие варианта 14:

Использовать `DataFrame`, содержащий следующие колонки: фамилия, имя; знак Зодиака; дата рождения (список из трех чисел). Написать программу, выполняющую следующие действия: ввод с клавиатуры данных в список; ввод с клавиатуры данных и добавление строк в `DataFrame`; записи должны быть упорядочены по датам рождения; вывод на экран информации о человеке, чья фамилия введена с клавиатуры; если такого нет, выдать на дисплей соответствующее сообщение.

`Main.py`:

```
import pandas as pd
import argparse
import os
from datetime import datetime

FILE_NAME = "data.parquet"
```



```

def load_data():
    if os.path.exists(FILE_NAME):
        return pd.read_parquet(FILE_NAME)
    else:
        return pd.DataFrame(columns=["last_name", "first_name", "zodiac",
"birth_date"])

def save_data(df):
    df.to_parquet(FILE_NAME, index=False)
    print("Data saved to data.parquet")

def add_person(df, last_name, first_name, zodiac, day, month, year):
    try:
        birth_date = datetime(year, month, day)
        new_row = pd.DataFrame([{
            "last_name": last_name,
            "first_name": first_name,
            "zodiac": zodiac,
            "birth_date": birth_date
        }])
        df = pd.concat([df, new_row], ignore_index=True)
        print(f"Added: {last_name} {first_name}")
        return df
    except Exception as e:
        print(f"Error: Invalid date - {e}")
        return df

```

```

def find_by_lastname(df, last_name):
    result = df[df["last_name"].str.lower() == last_name.lower()]
    if len(result) == 0:
        print(f"Not found: {last_name}")
    else:
        print("\nFound:")
        for _, row in result.iterrows():
            birth = row["birth_date"].strftime("%d.%m.%Y")
            print(f" {row['last_name']} {row['first_name']}, {row['zodiac']},
{birth}")

```

```

def show_sorted(df):
    if len(df) == 0:
        print("No data")
        return
    df_sorted = df.sort_values("birth_date")
    print("\nSorted by birth date:")
    for _, row in df_sorted.iterrows():
        birth = row["birth_date"].strftime("%d.%m.%Y")
        print(f" {row['last_name']} {row['first_name']} - {birth}
({row['zodiac']})")

```

```

def delete_column(df, column):
    if column in df.columns:
        df.drop(columns=[column], inplace=True)
        print(f"Column '{column}' deleted")

```

```

        print(f"Remaining columns: {list(df.columns)}")
    else:
        print(f"Column '{column}' not found")
    return df

def main():
    parser = argparse.ArgumentParser(description="Person database")

    parser.add_argument("--add", nargs=6, metavar=("LAST_NAME",
"FIRST_NAME", "ZODIAC", "DAY", "MONTH", "YEAR"),
                        help="Add a new person")

    parser.add_argument("--find", metavar="LAST_NAME", help="Find
person by last name")

    parser.add_argument("--show", action="store_true", help="Show all
persons sorted by birth date")

    parser.add_argument("--delcol", metavar="COLUMN_NAME",
                        help="Delete a column (e.g., zodiac)")

    args = parser.parse_args()

    df = load_data()

    if args.add:
        last_name, first_name, zodiac, day, month, year = args.add
        df = add_person(df, last_name, first_name, zodiac, int(day),
int(month), int(year))
        save_data(df)

```

```
if args.find:
    find_by_lastname(df, args.find)

if args.show:
    show_sorted(df)

if args.delcol:
    df = delete_column(df, args.delcol)
    save_data(df)

if not (args.add or args.find or args.show or args.delcol):
    parser.print_help()

if __name__ == "__main__":
    main()
```

Вывод: познакомились с основами работы с библиотекой pandas, в частности, со структурой данных DataFrame.